Snowflake DCM Projects

Declarative definition and lifecycle management
for Snowflake infrastructure, pipelines and governance

© 2026 Snowflake Inc. All Rights Reserved

For Snowflake Colleagues and Customers

Colleagues can copy or download

Do not publish or share without approval!

Safe Harbor and Disclaimers

Other than statements of historical fact, all statements contained in these materials and any accompanying oral commentary (collectively, the "Materials") are forward-looking statements within the meaning of Section 27A of the Securities Act of 1933, as amended, and Section 21E of the Securities Exchange Act of 1934, as amended, including statements regarding (i) Snowflake's business strategy, plans, opportunities, or priorities; (ii) Snowflake's new or enhanced products, services, and technology offerings, including those that are under development or not generally available; (iii) market size and growth, trends, and competitive considerations; (iv) Snowflake's vision, strategy, and expected benefits relating to artificial intelligence ("AI"), Snowpark, Snowflake Marketplace, the AI Data Cloud, and AI Data Clouds for specific industries or product categories, including the expected benefits and network effects of the AI Data Cloud; and (v) the integration, interoperability, and availability of Snowflake's products, services, or technology offerings with or on third-party platforms or products, including public cloud platforms. These forward-looking statements are subject to a number of known and unknown risks, uncertainties, and assumptions, including those described under the heading "Risk Factors" and elsewhere in the Annual Reports on Form 10-K and the Quarterly Reports on Form 10-Q that Snowflake files with the Securities and Exchange Commission from time to time. In light of these risks, uncertainties, and assumptions, the future events and trends discussed in the Materials may not occur, and actual results could differ materially and adversely from those expressed or implied in the forward-looking statements. As a result, you should not rely on any forwarding-looking statements as predictions of future events. Forward-looking statements speak only as of the date the statements are first made and are based on information available to Snowflake at the time those statements are made and/or management's good faith belief as of that time. Except as required by law, we undertake no obligation, and do not intend, to update the forward-looking statements in these Materials.

Any future product or roadmap information (collectively, the "Roadmap") is intended to outline general product direction. The Roadmap is not a commitment, promise, or legal obligation for Snowflake to deliver any future products, features, or functionality; and is not intended to be, and shall not be deemed to be, incorporated into any contract. The actual timing of any product, feature, or functionality that is ultimately made available may be different from what is presented in the Roadmap. The Roadmap information should not be used when making a purchasing decision. In case of conflict between the information contained in the Materials and official Snowflake documentation, official Snowflake documentation should take precedence over these Materials. Further, note that Snowflake has made no determination as to whether separate fees will be charged for any future products, features, and/or functionality which may ultimately be made available. Snowflake may, in its own discretion, choose to charge separate fees for the delivery of any future products, features, and/or functionality which are ultimately made available.

The Materials may contain information provided by third-parties. Snowflake has not independently verified this information, and usage of this information does not mean or imply that Snowflake has adopted this information as its own or independently verified its accuracy.

© 2026 Snowflake Inc. All rights reserved. Snowflake, the Snowflake logo, and all other Snowflake product, feature and service names mentioned in the Materials are registered trademarks or trademarks of Snowflake Inc. in the United States and other countries. All other brand names or logos mentioned or used in the Materials are for identification purposes only and may be the trademarks of their respective holder(s). Snowflake may not be associated with, or be sponsored or endorsed by, any such holder(s).

Accelerate with Confidence

One source of truth, reviewed before it lands

DCM Projects turns the configuration of a Snowflake account into version-controlled code, and puts a dry run in front of every deployment. Provisioning, governance changes, and data product releases all move through the same reviewed path.

- **Move faster**: one place to change and one place to check, instead of repeating the same edit per team, per account, per environment.
- **Deploy predictably**: see the effect of a change before you apply it, so errors that used to surface during deployment surface before it.
- **Stay traceable**: every deployment is recorded with the definitions that were applied and when, independently of the own source control.
- **Recover quickly**: the previous state is described in code, so restoring it is a deployment rather than a reconstruction.

Speed and predictability are the same problem at enterprise scale, because neither half is usable in production without the other.



Declarative Definitions



Preview all Changes



Reproducible Deployment History

© 2026 Snowflake Inc. All Rights Reserved

Business Impact

One framework – value for every team

Team	What changes for them	What the business gets
Platform engineers and DevOps	New teams provisioned from a template entry. Cross-team changes made once. Every change reviewable before it lands.	Shorter provisioning turnaround for the teams waiting on them, fewer failed changes, less load on a team that is usually small relative to demand
Admins and data stewards	Policies, privileges, and external access defined in one reviewable place, with the current state readable rather than reconstructed.	Faster access turnaround, consistent standards across accounts, a defensible audit trail, faster recovery from an unintended privilege change
Data and analytics engineers	Data products versioned and recreatable. Changes validated against real data before release. Every environment produced from one source.	Fixes and improvements reach production sooner on the assets the business consumes

Each team can run its own project, with the same review path, audit trail, and multi-environment model available in every one of them.

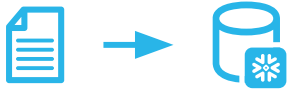
© 2026 Snowflake Inc. All Rights Reserved

In Dev Private Public GA

Snowflake DCM Projects

WHAT IS IT

- Infrastructure-as-code with declarative SQL definitions
- Parameterized infrastructure templates for large organizations or asset bundles for data products / pipelines
- A CI/CD solution for deploying changes to Snowflake



WHY USE IT

- Keep all infrastructure defined as code - versioned in git
- Test infrastructure changes before deploying them to production
- Preview what will change with each deployment
- Easily manage different environments
- Natively integrated in Snowflake (leveraging RBAC, Horizon, Cortex Code, snowCLI, Workspaces, etc)

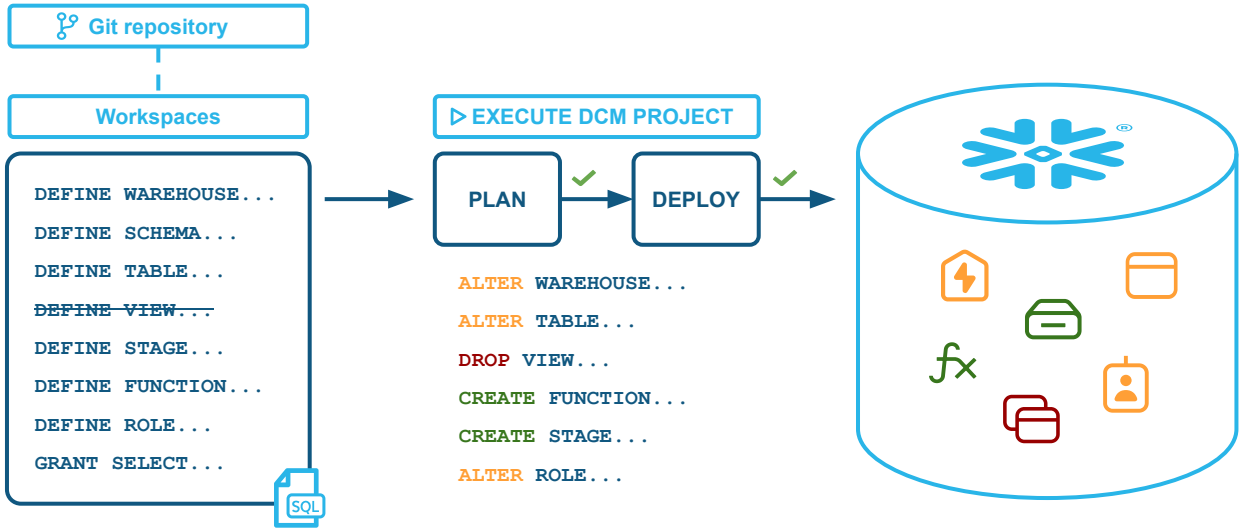
HOW TO USE IT

- Create a schema-level “DCM Project” object that can be executed to PLAN and DEPLOY these definitions to a target environment
- Flexibly organize your object definitions in SQL files
- Define your target environments and values for templating variables in a project manifest.yml
- Let Cortex migrate your existing infrastructure or create, debug and extend new data product with you

© 2026 Snowflake Inc. All Rights Reserved

Snowflake DCM Projects

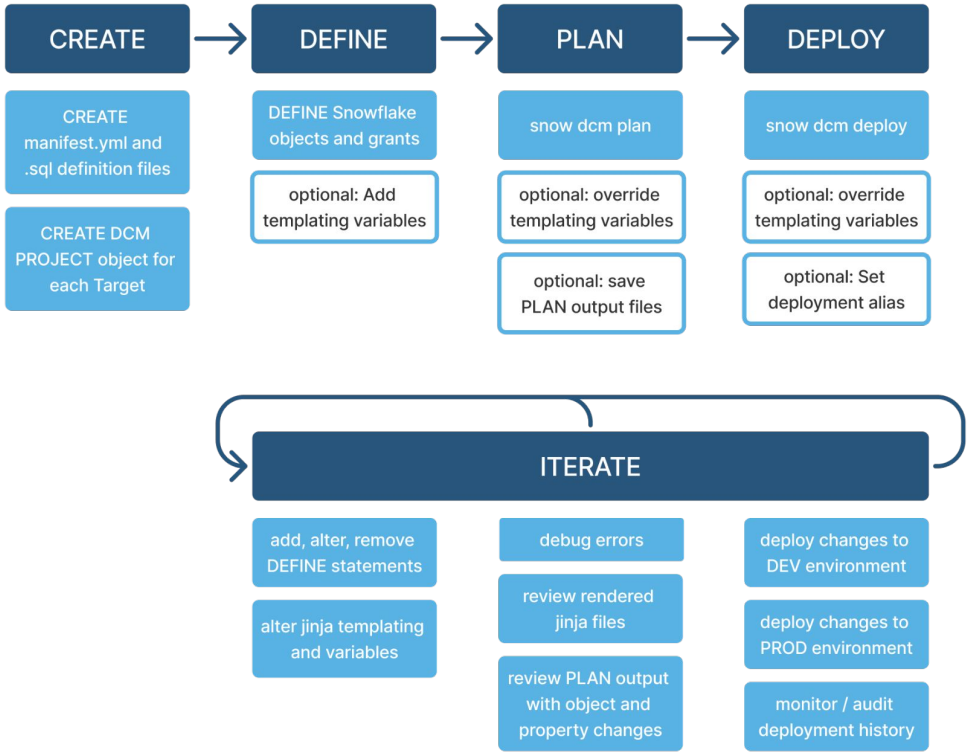
Declarative definition and lifecycle management
for Snowflake infrastructure, pipelines and governance



© 2026 Snowflake Inc. All Rights Reserved

DCM Projects Lifecycle

- Multiple users can iterate over the same project definitions to continuously extend and evolve a data product or account template
- Each change can be planned (dry-run) and deployed incrementally
- Project definitions can be deployed manually to an isolated DEV sandbox or a shared QA / Stage environment
- Deployments to production environments can be triggered as part of CI/CD pipelines or manually
- All incremental changes are preserved as immutable artifacts inside the DCM Project



© 2026 Snowflake Inc. All Rights Reserved

DCM Key Use Cases

DCM for managing Infrastructure



- Programmatically define, deploy and modify Snowflake infrastructure for large organizations in one place
- Define Databases, Schemas, Warehouses, Roles and Grants for various internal teams
- Leverage Jinja and macros to maintain one template for multiple teams - change it in one place and apply it to all
- List all teams and their individual configurations in one file

DCM for Data Pipelines



- Build, test and deploy transformation Pipelines using Dynamic Tables, Task Graphs or dbt Projects (PrPr)
- Version pipeline definitions as code and parameterize deployments to multiple environments
- Clone existing Dynamic Table / Task Graph / dbt projects into a dev-sandbox to iterate and test changes safely with real data before deployment

DCM for Data Governance



- Full audit trail for all infrastructure changes managed by DCM
- Manage (and revoke) grants programmatically at scale
- Rollout policy changes across accounts
- Define data quality gates for your deployment pipelines
- Drift detection: Automatically detect and alert on changes in role privileges, policies or data expectations

DCM for ML Infrastructure & AI Projects *(in preview)*



- Build, test and deploy Cortex Agents, Semantic Views, Search Services, Models, and Feature Stores
- Version your AI / ML in code and parameterize deployments to multiple environments
- Clone existing Agents, Semantic Views, and Models into a dev-sandbox to test changes safely before deployment

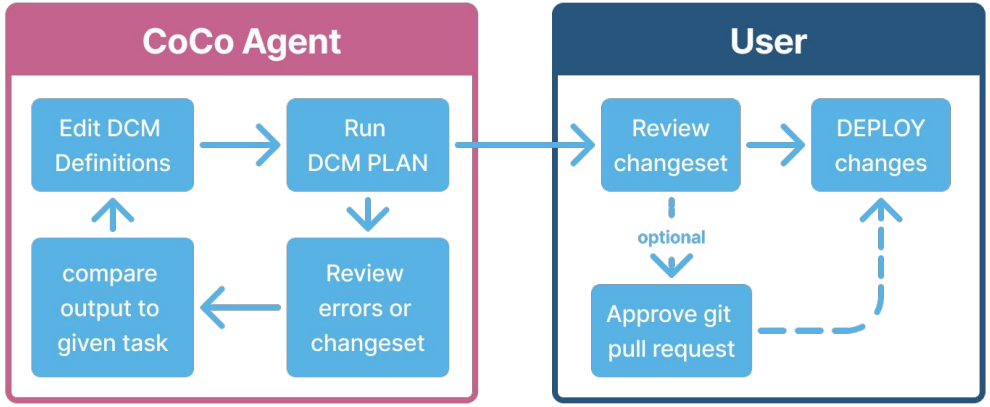
For all use cases CoCo agents can iterate autonomously and you review the plan before deployment

© 2026 Snowflake Inc. All Rights Reserved

Built for AI-assisted Data Engineering

Coding agents work in code

As coding agents take on more data engineering work, the constraint is how much supervision each agent-authored change needs before anyone will deploy it, rather than the agent's ability to write code.



- **An account described in code is an account an agent can read.** Without it, an agent has to interrogate the live account to work out the current state, which costs cycles and requires deep knowledge of the Snowflake database schema. With DCM definitions, the current state is already in the form the agent works in, so its effort are focused on the requested changes.
- **The dry run gives the agent a feedback loop it can close by itself.** It proposes a change, examines what that change would do, sees the errors and side effects, and iterates until the result matches the intent. That loop runs without a person in it.
- **What reaches the reviewer has already been validated.** That is a lighter review task than checking unverified output.

Making changes with CoCo is easy. Deploying them with DCM is predictable. The combination makes it usable in production.

© 2026 Snowflake Inc. All Rights Reserved

DCM Key Capabilities

DCM Skills bundled in **Cortex Code** (CLI & Workspaces)

- **Declarative Definitions**
  Define the target state of your infrastructure independently of its current state
- **Jinja Templating**
  Leverage macros and functions for scalable templating and less redundancy
- **Data Preview** (in Preview)
  See sample data for Views and Dynamic Tables based on their current definitions - even before they are deployed
- **Auto Dependency Resolution**
  Organize your object definitions freely across files and DCM will always execute them in the right order
- **snowCLI Commands**
  Short and powerful commands for local editing and CI/CD automation
- **PLAN command**
  Dry-run a deployment of a new project definition to see what would change
- **Workspace UI support**
  Create, preview, test, plan and deploy DCM Projects in a Snowflake Workspace without having to write SQL commands
- **Data Tests** (in preview)
  Check all data quality expectations attached to tables inside the project to verify integrity of transformation logic
- **Deployment History**
  Definition snapshots of each deployment are stored automatically and immutable inside the DCM project for full auditability of infrastructure changes
- **Data Lineage Graph**
  See dependencies between table objects based on their definitions - even before they are deployed

© 2026 Snowflake Inc. All Rights Reserved



DCM Projects for Infrastructure-as-code

Managing Snowflake account infrastructure at scale

© 2026 Snowflake Inc. All Rights Reserved

Common Challenges for Data Platform Teams



Inconsistent state and schema drift across environments



Administrative overhead of managing setup and change requests at scale



Cost and complexity of integrating and maintaining third-party tools



Risky manual or ad-hoc production deployments

What changes with DCM Projects

All environments are built from the same definitions, so pre-production testing is trustworthy

A new team is a manifest entry and a deploy, and a cross-team change is one template edit

A first-party capability inside the platform, with nothing extra to procure, secure, or maintain

PLAN shows the full changeset before anything is applied, and deployment runs through the CI/CD pipeline

© 2026 Snowflake Inc. All Rights Reserved

DCM for Snowflake Infrastructure

In Dev · Private · Public · GA

One template, many teams
Define databases, schemas, warehouses, roles, and grants as parameterized Jinja templates. Add a team entry to the manifest dictionary to deploy the full infrastructure set. Remove the entry to drop it.

Cross-team changes in one edit
Add a read-only role, change warehouse sizes, or update retention policies across all teams by editing the shared template once. PLAN shows the full delta across every team before deploying.

Configuration enforcement
Run PLAN periodically to compare current state against defined standards. If someone altered a warehouse size or changed a grant outside DCM, PLAN detects the drift. Re-deploy to reinstate.

Ownership delegation
Platform team deploys account-level infrastructure and grants CREATE DCM PROJECT to team leads. Each team manages their own objects within the provisioned database using their own project.

Multi-environment from one codebase
Same definitions deploy to dev, staging, and production. Environment-specific configs control warehouse sizes, retention policies, and naming prefixes.

```yaml
templating:
  defaults:
    users:
      - "GITHUB_ACTIONS_SERVICE_USER"
    wh_size: "X-SMALL"

  configurations:
    DEV:
      env_suffix: "_DEV"
      users:
        - "GITHUB_ACTIONS_SERVICE_USER"
        - "DEV_1"
      project_owner_role: "DCM_DEVELOPER"
      teams:
        - name: "Dev Team"
          raw_access: "WRITE"
        - name: "Test Team 2"
          raw_access: "READ"

    PROD:
      env_suffix: ""
      project_owner_role: "DCM_PROD_DEPLOYER"
      wh_size: "MEDIUM"
      teams:
        - name: "Marketing"
          raw_access: "READ"
        - name: "Finance"
          raw_access: "READ"
        - name: "HR"
          raw_access: "NONE"
        - name: "IT"
          raw_access: "WRITE"
        - name: "Sales"
          raw_access: "NONE"
        - name: "Research"
          raw_access: "NONE"
        - name: "Design"
          raw_access: "NONE"
```

© 2026 Snowflake Inc. All Rights Reserved



DCM Projects for Data Pipelines

Easier editing and more reliable testing and deployment

© 2026 Snowflake Inc. All Rights Reserved

Common Challenges for Analytics & BI-Teams



DevOps tooling and lifecycle processes add too much complexity for SQL-focused business teams



Long waits for admins to provision access and infrastructure



Pipeline debugging and ad-hoc configuration changes are slow and error-prone



No integrated version control, collaboration, or cross-dependency awareness

What changes with DCM Projects

Definitions are Snowflake SQL, authored in Workspaces or through Cortex Code, with no external toolchain to adopt

Platform teams delegate project ownership, so teams manage their own objects inside the space provisioned for them

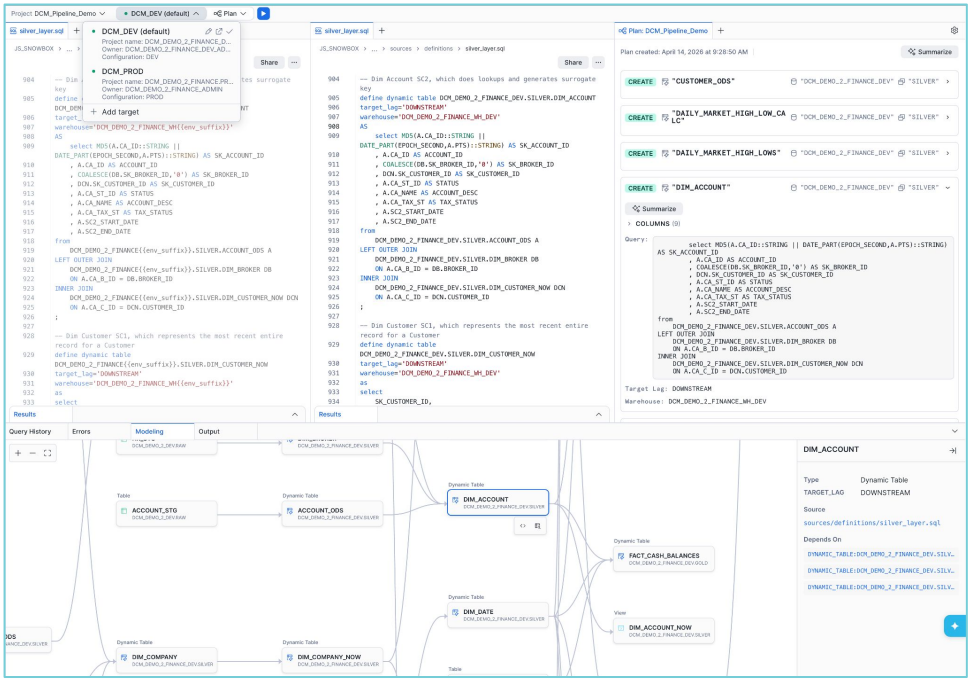
Changes are previewed against real data and planned before deployment, so problems are found ahead of release

Git in Workspaces, dependency order resolved automatically, and a lineage graph available before deployment

© 2026 Snowflake Inc. All Rights Reserved

DCM Projects for Dynamic Tables

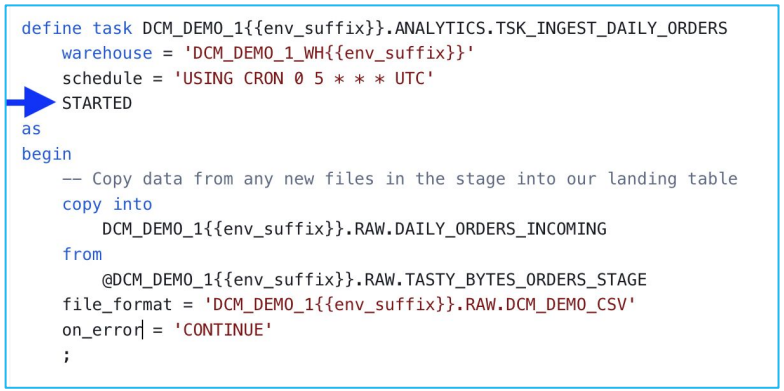
- **PLAN your pipeline changes**
  See the exact changeset before deploying. Configuration changes (target lag, warehouse, transformation logic) surface as a precise diff. Errors caught before deployment.
- **PREVIEW with real data**
  Get a data sample from a changed DT definition without deploying. Validate whether a new column or modified join produces expected results before anything touches production.
- **TEST as CI/CD gates**
  DCM's TEST command evaluates data quality expectations. Deploy → run/refresh → test → promote or fail.
- **Visual dependency graph**
  Bronze → silver → gold ordering resolved automatically. Column-level lineage visible in Workspaces.
  Click a node, jump to the code.
- **Multi-environment from one codebase**
  Same definitions deploy to dev, staging, and production via Jinja-templated variables (parameterized database names, warehouse sizes, target lags).



© 2026 Snowflake Inc. All Rights Reserved

DCM for Tasks & Task Graphs

- **Define the target state**
  Each task in a graph can declare STARTED or SUSPENDED directly in the DEFINE statement. No post-deployment scripts needed to resume tasks. DCM resolves root-task and child-task state ordering automatically.
- **Target state enforced on every deploy**
  If a task is defined as STARTED and someone suspends it manually outside DCM, the next deployment starts it again. (or vice-versa)
  The definition is the source of truth.
- **Auto-suspend/resume during changes**
  Running tasks are automatically suspended before applying definition changes, then resumed after. No manual intervention, no missed schedule windows from forgetting to resume.
- **PLAN the full graph**
  See every change across the task graph (schedules, warehouses, task bodies, predecessor relationships) as a precise diff before anything is applied. Errors caught before deployment.
- **Parameterize task properties**
  Use Jinja variables for retry attempts, timeout, warehouse size, and other task properties. Change the templating config once, apply to all tasks.
  Or use SUSPENDED in dev profiles and STARTED in production.



```
define task DCM_DEMO_1{{env_suffix}}.ANALYTICS.TSK_INGEST_DAILY_ORDERS
    warehouse = 'DCM_DEMO_1_WH{{env_suffix}}'
    schedule = 'USING CRON 0 5 * * * UTC'
    STARTED
as
begin
    -- Copy data from any new files in the stage into our landing table
    copy into
        DCM_DEMO_1{{env_suffix}}.RAW.DAILY_ORDERS_INCOMING
    from
        @DCM_DEMO_1{{env_suffix}}.RAW.TASTY_BYTES_ORDERS_STAGE
    file_format = 'DCM_DEMO_1{{env_suffix}}.RAW.DCM_DEMO_CSV'
    on_error = 'CONTINUE'
    ;
```

© 2026 Snowflake Inc. All Rights Reserved



DCM Projects for Data Governance

Managing access privileges and policies at scale

© 2026 Snowflake Inc. All Rights Reserved

Common Challenges for Admins and Data Stewards



Privileges and policies maintained by hand per account and per team, so standards diverge



Answering what a role can reach today means inspecting the account, privilege by privilege



Audit evidence assembled after the fact from tickets, scripts, and recollection



A privilege changed outside the process stays invisible until it causes a problem

What changes with DCM Projects

Define roles, grants, and policies once as templates, and change them everywhere with one edit

The deployed definitions are the current state, and PLAN shows the effect of a change before it applies

Every deployment is recorded in the account with all details, queryable and linkable to its approval ticket

PLAN compares the account against the defined standard, and re-deploying reinstates it

© 2026 Snowflake Inc. All Rights Reserved

DCM for Data Governance

Comprehensive Audit Trails & Observability:

- Maintains a centralized repository with immutable snapshots of all deployment artifacts.
- Allows monitors to review deployment history and object-level changes via Snowsight or by querying INFORMATION_SCHEMA.DCM_DEPLOYMENT_HISTORY().

Declarative Privilege Management at Scale:

- Defines complex RBAC structures, database roles, and exact privilege assignments (GRANTS) as code.
- Leverages Jinja2 templating (macros, default values, and profiles) to efficiently scale grant templates across schemas and accounts.

Infrastructure & Privilege Drift Detection:

- Identifies state divergence by comparing defined infrastructure in code against the actual account state.
- Utilizes the PLAN (dry-run) command or scheduled cron-jobs to detect manual alterations or revoked privileges.

Strict Project Ownership & Separation of Concerns:

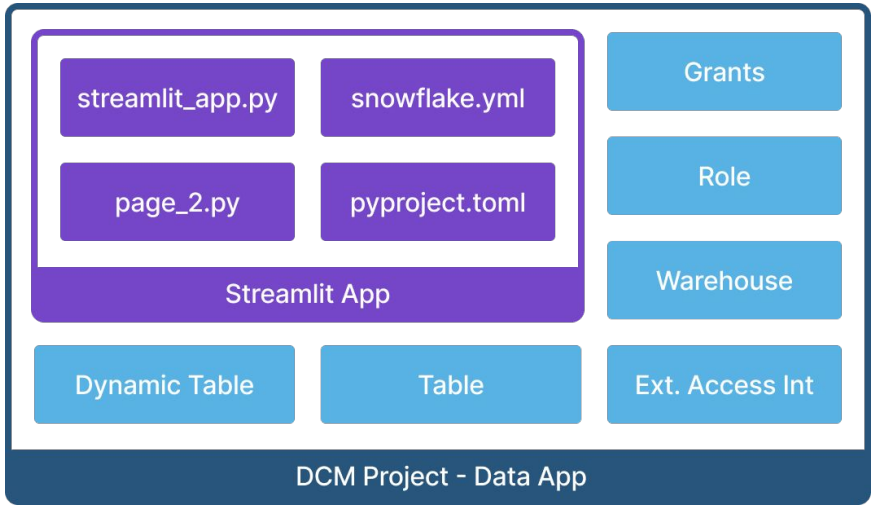
- Enforces standard RBAC logic, requiring dedicated "deployer-roles" with tailored privileges to alter account infrastructure.
- Isolates access control by assigning different DCM projects to different owner roles (e.g., USER_ADMIN for grants, SYSADMIN for databases).

© 2026 Snowflake Inc. All Rights Reserved

In Dev Private Public GA

DCM for Streamlits (Public Preview)

- **Bundle your Streamlit app** together with other required Snowflake objects into one DCM project
- Just reference your existing Streamlit folder as a **DCM asset** (via a relative path) and then name the asset in your DEFINE STREAMLIT definition
- **Easily deploy** your existing Streamlit to **different target environments** so you can test changes on a DEV instance before deploying to PROD
- Only deploy new versions when Streamlit files have changed



© 2026 Snowflake Inc. All Rights Reserved

Interfaces for DCM Projects

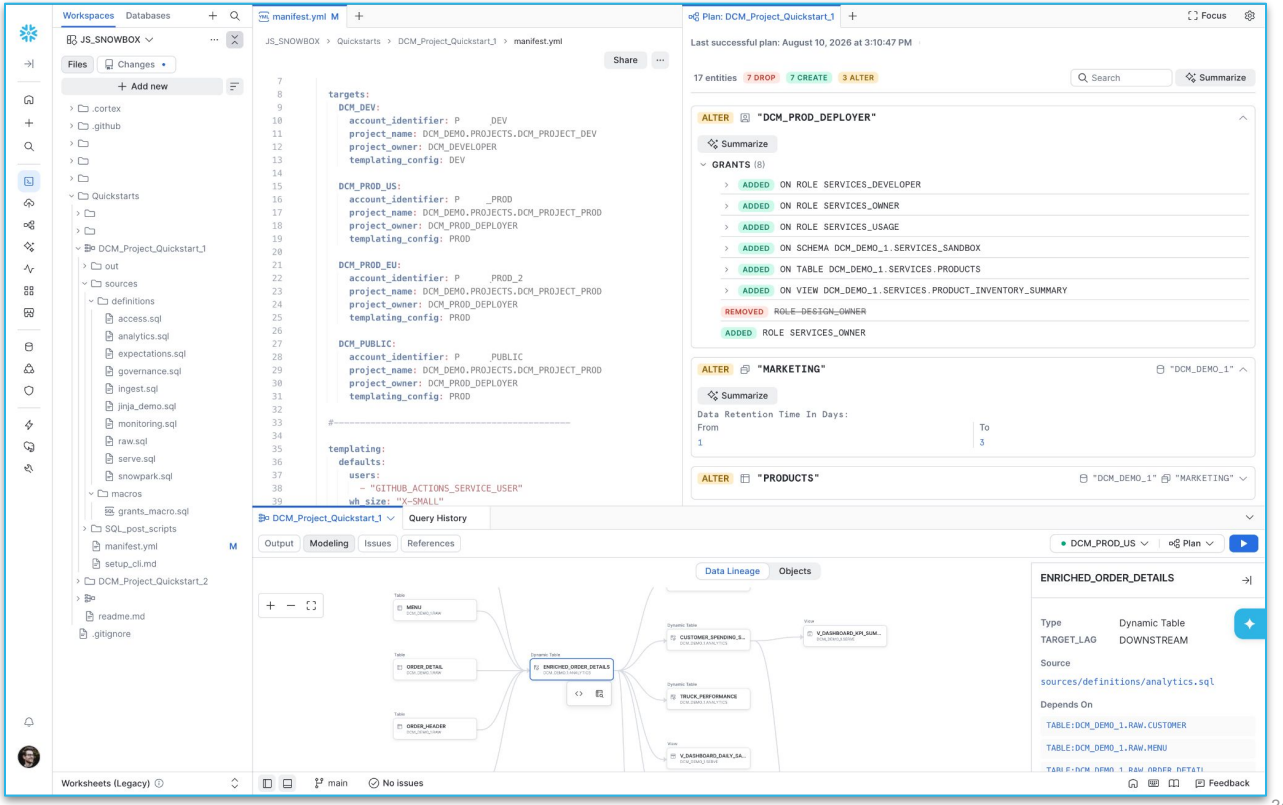
Snowsight Workspaces or snowCLI
both with support from Cortex Code

© 2026 Snowflake Inc. All Rights Reserved

Snowsight Workspaces

Integrated cloud IDE:

- Easily manage DCM project files
- Inspect current project state, edit project definitions and run DCM commands all in one place
- Rich PLAN output UI with drill-down option
- Table Lineage DAG linking to definition code
- Connect to remote git repository to fetch and commit changes
- Collaborate using git branches
- Simply log in to your Snowflake account to get started

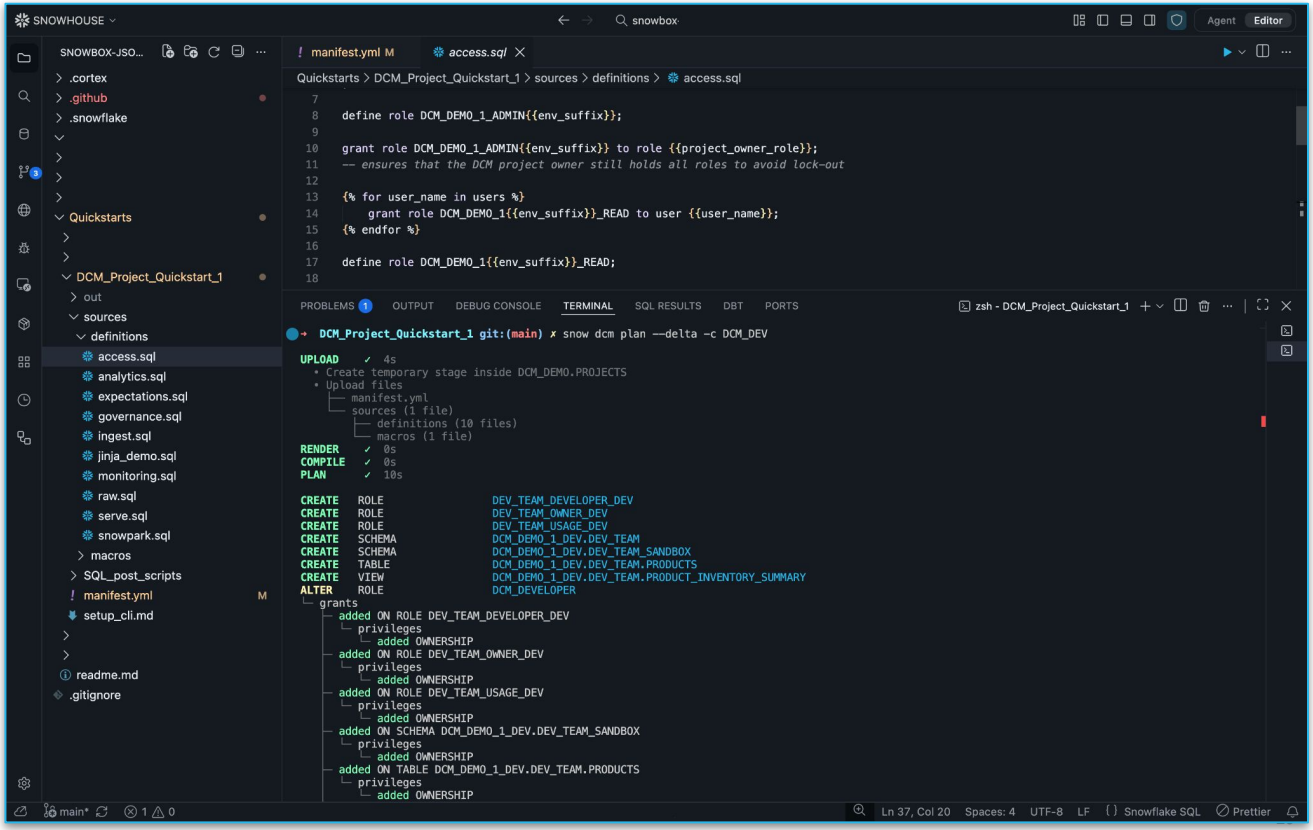


© 2026 Snowflake Inc. All Rights Reserved

snowCLI Commands in Local Editors

Your local IDE of choice:

- Easily manage your Snowflake account connections in yaml
- Clone your project files from remote repository to your local disk
- Leverage snowCLI commands for DCM to manage projects, PLAN and DEPLOY changes
- Collaborate using git branches

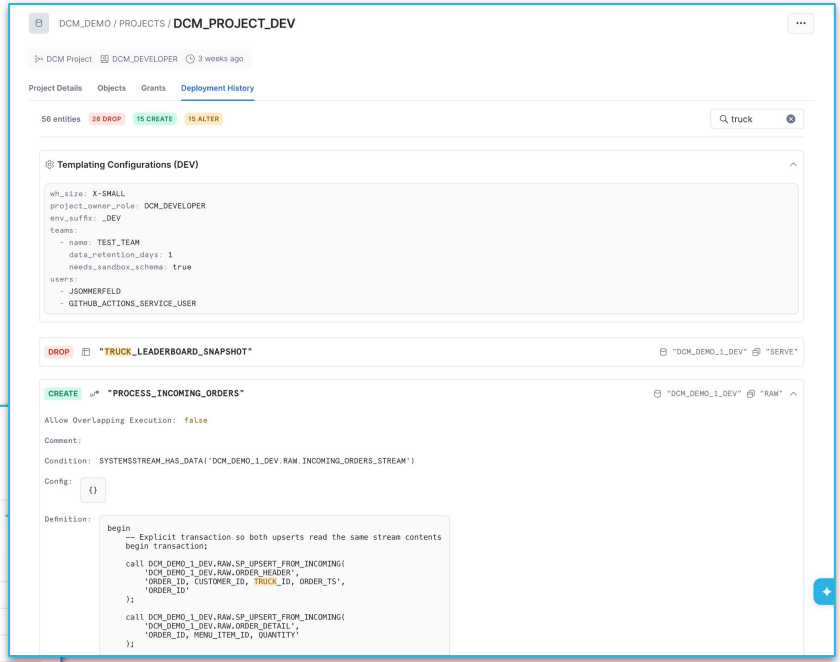


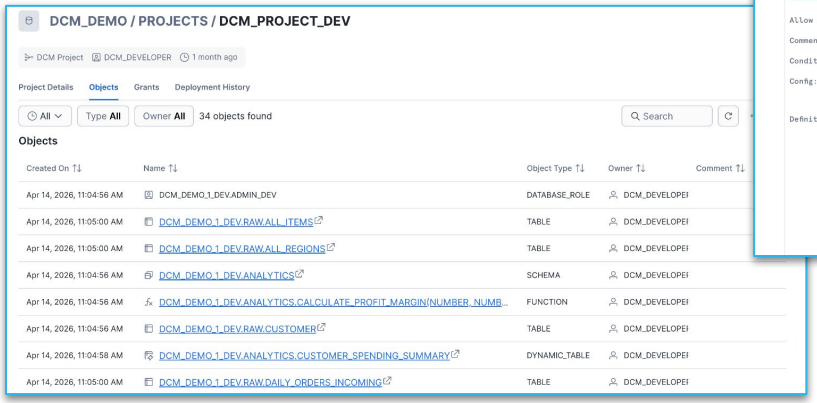
© 2026 Snowflake Inc. All Rights Reserved

Snowsight Observability

DCM Projects in the Horizon Catalog

- See existing DCM Project objects in each schema and review or alter role privileges via the UI
- See all objects and grants managed by the DCM project
- Deployment History shows which changes were made when
- Deployment Details show configurations and all individual changes made during that deployment





© 2026 Snowflake Inc. All Rights Reserved

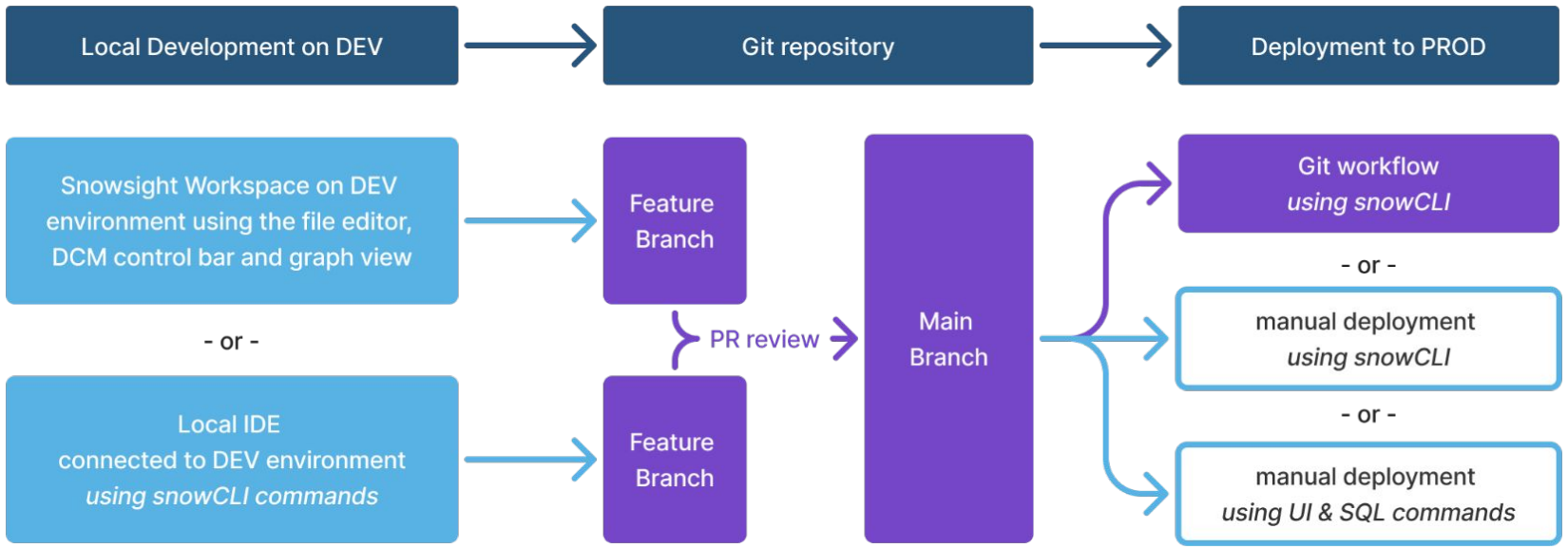


CI/CD with DCM Projects

“Outer loop” experience with deployment pipelines

© 2026 Snowflake Inc. All Rights Reserved

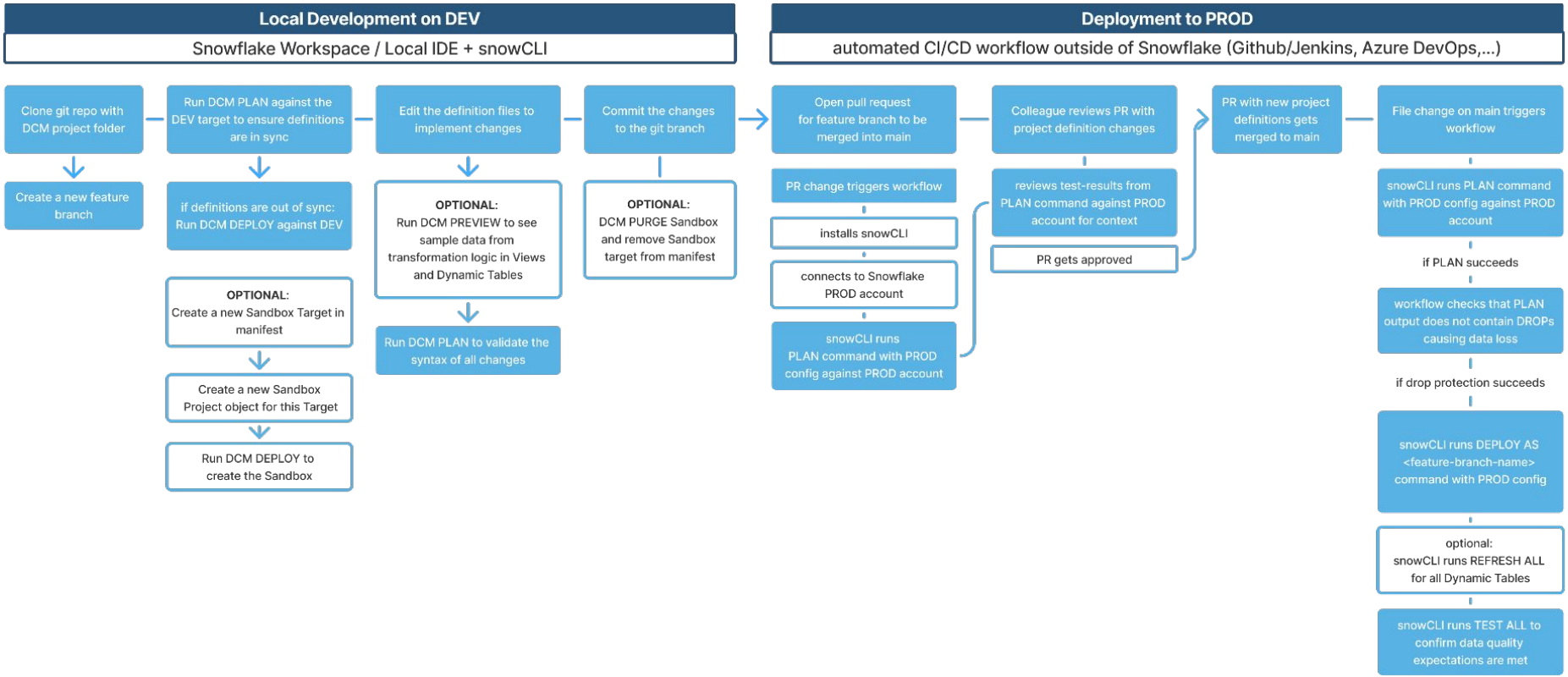
DCM Project Collaboration & Deployment



© 2026 Snowflake Inc. All Rights Reserved

In Dev Private Public GA

DCM Project CI/CD flow



© 2026 Snowflake Inc. All Rights Reserved

CI/CD with DCM Projects

Run Github Actions for DCM From the Github Marketplace

- **Keep DCM definitions in Git** — store manifest.yml and SQL DEFINE files in the repo, use feature branches and pull requests, and treat Snowflake infrastructure changes like application code.
- **Use Snowflake CLI in GitHub Workflows** — a typical workflow checks out the repo, installs snowflakedb/snowflake-cli-action, and runs snow dcm plan --target ... --save-output for automated validation.
- **Make PR review easier with PLAN output** — run PLAN on every pull request so reviewers see the exact production impact before anything is deployed.
- **Map GitHub environments to manifest targets** — for example DCM_STAGE and DCM_PROD_US; keep account identifier and project owner in the version-controlled manifest.yml, while storing only service-user credentials as GitHub secrets.
- **Validate all targets automatically** — use a matrix job to test connectivity and configuration for every target environment in parallel when onboarding a new repo or account.
- **Deploy on merge with controlled promotion** — after approval and merge to main, trigger snow dcm deploy to promote the same definitions into higher environments or production.

** The pattern is not GitHub-specific — the same approach applies to Azure DevOps, GitLab CI/CD, or Bitbucket etc; only the workflow syntax changes*

© 2026 Snowflake Inc. All Rights Reserved

GitHub Actions for DCM Projects

Set of reusable composite GitHub Actions that automate DCM Projects pipelines

Now officially published on the GitHub Marketplace *(public preview)*

Referenced in GitHub workflows as
`- uses: snowflakedb/snowflake-actions/dcm/<action-name>@v3`

Action	Description
`dcm-parse-manifest`	Parses `manifest.yml` and outputs target names as a JSON array for matrix strategies.
`dcm-connection-test`	Tests Snowflake connectivity, validates that the connection role matches the manifest `project_owner`, and checks whether the DCM project object already exists.
`dcm-plan`	Runs `snow dcm plan`, summarizes the changeset (CREATE, ALTER, DROP counts by object domain), and uploads plan artifacts. Optionally runs as PLAN DELTA. Optionally posts the plan summary as a comment on the associated pull request.
`dcm-deploy`	Deploys the DCM project with optional data drop detection, and post-deployment SQL scripts. Optionally posts a deploy summary to the pull request.



Getting started with DCM Projects

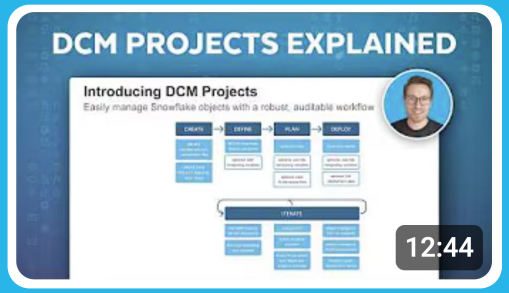
Public resources to try out DCM today

© 2026 Snowflake Inc. All Rights Reserved

Getting Started

Published resources

- **Developer Guides** (Quickstarts)
  - Get Started with Snowflake DCM Projects
  - Build Data Pipelines with Snowflake DCM Projects
  - DCM-Projects for Dynamic Tables
  - DCM Projects for Tasks
- **Public DCM git repository under Snowflake-labs** *(frequent updates)*
  https://github.com/Snowflake-Labs/snowflake-dcm-projects
  - Demo projects
  - Sample Github workflows
- **Documentation**:
  - Snowflake DCM Projects
  - DCM Projects early-access (PrPr)
  - DevOps with Snowflake
- **Recommended Blog posts:**
  - https://select.dev/posts/snowflake-dcm-projects-infrastructure-as-code-in-plain-sql
  - A CI/CD Pipeline with Snowflake DCM Projects
  - Terraform for Snowflake, But Without Terraform
  - Declarative Change Management Without the Script Sprawl
  - DCM Projects: A Quiet Revolution in DataOps



📺 **Introduction to DCM Projects Declarative Infrastructure Management (with Demo)**

📺 **Demo Video**
CoCo Skills for DCM Projects

📺 **Demo Video**
DCM for dbt projects (early access)

© 2026 Snowflake Inc. All Rights Reserved



Importing Existing Infrastructure

Migration tools to bring existing Snowflake objects into DCM Projects

© 2026 Snowflake Inc. All Rights Reserved

Importing existing infrastructure into DCM

The Snowflake-Labs repository contains a **Python Procedure** to generate object definitions and a second procedure to generate roles and grants.
An additional **SKILL.md for Cortex Code** is available to assist users with migrating existing Snowflake objects into a DCM project. At its core this process runs `GET_DDL()` for all supported objects within a given schema or database and writes them as DEFINE statements into .sql files.

The Cortex Code skill wraps this generation step in a **guided workflow**:

- It executes the python procedure against the selected database or schemas
- produces the DEFINE files
- runs PLAN to compare them against the live objects
- reviews the diff to refine the DEFINE statements where needed so they match the current state
- then runs DEPLOY with zero-changes to adopt the objects into DCM management
- optionally identify Jinja templating opportunities for multi-environment setups

The agent pauses at key checkpoints for review and approval.

The stored procedure is the fallback when Cortex Code is not available. It covers the file generation step on its own, with project setup, PLAN, and DEPLOY run manually afterwards.

ℹ️ **Reference the skill** or copy the procedure from our **public Snowflake-Labs repository**:
https://github.com/Snowflake-Labs/snowflake-dcm-projects/tree/main/migration-tools/

⚠️Both of these are currently shared as a "**Public Preview**" for testing and to be used at your own risk.

© 2026 Snowflake Inc. All Rights Reserved

DCM Projects
Early access (PrPr)

Rolling set of additional DCM capabilities available as private preview to early access customers

© 2026 Snowflake Inc. All Rights Reserved

DCM Projects - early access (PrPr)

Current set of capabilities in early-access

- **DEFINE DBT PROJECT**
  - Define a dbt project, its orchestration, infrastructure, and access control together in a single DCM project folder - then deploy everything to any environment.
- **DEFINE SECRET & env-vars**
  - Inject keys and passwords at deploy-time from your environment (vault) to create or update Snowflake secrets
  - Without exposing them in your code or logs
- **DEFINE TASK GRAPH** (Task Graph is also in PrPr)
- **DEFINE AGENT**
  - From inline yaml or .yaml-file as DCM asset
- **DEFINE EXTERNAL ACCESS INTEGRATION**
- **DEFINE STORAGE INTEGRATION**
- **DEFINE API INTEGRATION**
- **Improved CLI commands for DCM**
  - New `snow dcm init` command to set up a new DCM Projects from 0 to 1
  - New `snow dcm compile` command for quickly checking for errors
  - Experimental `snow dcm dependencies` command to render object dependency graph in markdown

ℹ️ *New DCM capabilities will be released either straight to PuPr or first to customers participating in the DCM early access PrPr. (based on feature maturity) Early-access customers will automatically have access to a rolling set of new capabilities. ->* early-access documentation

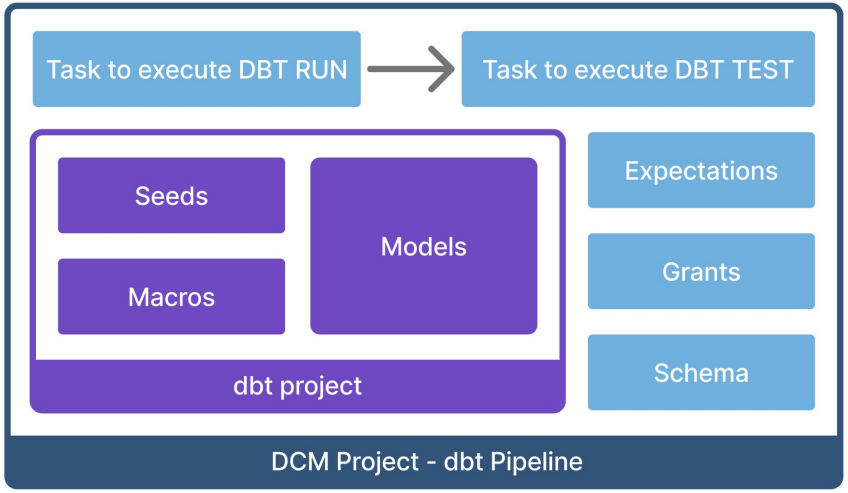
© 2026 Snowflake Inc. All Rights Reserved

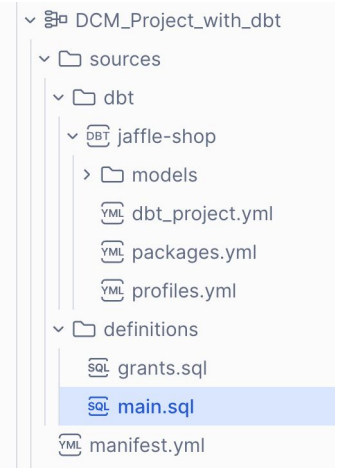
In Dev | **Private** | Public | GA

DCM for dbt-projects (early-access PrPr)

- Package your dbt-project together with all other required Snowflake objects into one DCM project
- **Safely deploy** your existing dbt-projects to **different target environments**
- Only deploy new versions when dbt-definitions have changed
- No schema migration needed. Just **run your existing dbt models** as they are

📺 **Demo Video**





© 2026 Snowflake Inc. All Rights Reserved

DCM Projects
Product Roadmap

Planned additional DCM capabilities
- timeline and priorities are subject to change -

© 2026 Snowflake Inc. All Rights Reserved

DCM Improvements Roadmap

Performance Improvements

- Incremental deployments (non-linear execution times for larger projects)
- Faster plan delta and full plan

Scalability

- Gradually raising the size limits for DCM Projects from 10k to 100k entities

Fan-out deployments

- Parallel independent deployments to large number of project targets for “multi-tenant” projects

SQL posthook

- Executing a user-defined SQL script after every deployment (not part of plan!)
- To run CREATE IF NOT EXISTS / CREATE OR REPLACE, INSERT, EXECUTE etc. for any unsupported DDL or DML use-cases
- Supporting jinja-templating from the manifest.yml
- Stored as part of DCM deployment history artifacts

© 2026 Snowflake Inc. All Rights Reserved

Supported Entities

Available in GA

- Database
- Schema
- Table
- View
- Dynamic Table
- Task
- Stream
- Pipe
- Alert
- Procedures (all languages)
- Functions (all languages)
- Semantic View
- Data Metric Functions
- Tags
- Internal & external Stages
- File format
- Sequence
- Warehouse
- Role / Database Role
- Network Rule
- Network Policy
- Authentication Policy
- Share
- Grants
- Attach DMF

Available in Public Preview

- Streamlit
- Code Bundle (incl. Notebook, Spark)
- Masking Policy
- Row Access Policy
- ATTACH Tag
- ATTACH Masking Policy
- ATTACH Row Access Policy

Available in DCM Early Access (PrPr)

- Cortex Agent
- Iceberg Table (Snowflake-managed)
- Secret
- Task Graph
- API Integration
- External Access Integration
- Storage Integration
- dbt Project

In Development

- Resource Monitor
- Compute Pool
- Cortex Search Service

© 2026 Snowflake Inc. All Rights Reserved



3rd party Product Comparisons

© 2026 Snowflake Inc. All Rights Reserved

DCM Projects + Terraform

	DCM Projects	Terraform (Snowflake Provider)
Primary audience	Data engineers, analytics engineers, platform teams	DevOps engineers, platform teams
Object focus	Schema-level: Tables, Views, Dynamic Tables, Tasks, Procedures, etc. (+ Roles, Grants, Warehouses)	Account-level: Accounts, Users, SSO, Integrations, Policies, Network rules, Roles, Grants, Warehouses
Authoring	SQL-based DEFINE statements with Jinja	HCL configuration language
Execution	Inside Snowflake (server-side, batch operations)	Outside Snowflake (SQL via provider, per-resource queries)
State management	No state-file. DCM checks account state at deploy-time	State file as source of truth from latest deployment
Multi-environment	Manifest targets with Jinja configuration profiles	Terraform modules (general-purpose, not Snowflake-specific)
Snowflake UI	Native in Snowsight, Workspaces, and Cortex Code	None
Scope beyond Snowflake	Snowflake only	Any infrastructure with a Terraform provider
Validate role privileges	PLAN validates all required privileges	Deployment can fail due to insufficient privileges
Release status	GA (but some new entities in preview)	GA (but many new entities in preview)
Owner Role	One owner role only, but can grant ownership to lower roles	Define owner role for each individual entity

General recommendation: Co-existence
Terraform for managing account-level infrastructure and cross-cloud resources. DCM Projects for managing schema-level objects and data pipelines. Both tools operate in the same account without conflict. For customers starting fresh without Terraform experience, DCM covers most schema-level needs directly.

Limitation:
DCM does not manage users, SSO, external integrations or account creation.

© 2026 Snowflake Inc. All Rights Reserved

DCM Projects vs schemachange

Positioning:
DCM works fundamentally different than schemachange. Specific use-cases can benefit from shifting to DCM Projects declarative solution. But each solution has its pros & cons and not all schemachange use-cases will be covered by DCM Projects.

	DCM Projects	schemachange
Approach	Declarative = define target state, Snowflake resolves all operations	Imperative = ordered migration scripts, applied sequentially
State & drift	Server-side state in project object. PLAN detects drift on every run.	CHANGE_HISTORY table tracks applied scripts. No drift detection.
Plan / Dry-run	PLAN shows full changeset before deploying	VERIFY lists pending scripts, no desired-vs-current comparison
Data Pipeline lifecycle	Preview, Plan, Deploy, Test expectations	Not available
Templating & environments	Jinja2 with macros, manifest targets, configuration profiles	Jinja2 with macros, modules folder, variables per deployment
Snowflake integration	Native in Workspaces (visual editor, graph, one-click deploy), Cortex Code skills	None. Runs externally (Python, client-side SQL splitting).
Object removal	Remove a DEFINE, next deploy drops the object	Write a new versioned script to reverse the change
Maintenance & cost	Official product, currently in public preview, continuous development and expansion	Community-maintained, free, no official support
Strength	**Operational simplicity & predictability**	**Execution flexibility & customization**

© 2026 Snowflake Inc. All Rights Reserved

DCM Projects vs dbt Projects

Separation of responsibilities: dbt remains the modeling/ transformation engine to run DML statements, while DCM manages all DDLs for schemas, grants, policies, tags, semantic views, warehouses, and tasks for orchestration

	DCM Projects	dbt Projects (in Snowflake)
Primary audience	Data engineers / platform teams	Data product / mart / warehouse teams
Object focus	Continuously expanding range of schema-level and account-level entities as well as roles and grants	Data transformation pipelines
Authoring	Jinja-templated SQL language	Jinja-templated SQL language
Execution	Inside Snowflake (server-side, batch operations)	Outside and inside Snowflake (SQL)
Purpose	Deployment of Snowflake objects and schema changes	Running data transformation
Multi-environment	Manifest targets with Jinja configuration profiles	Using profiles and targets, separating configuration from code
Snowflake UI	Native in Snowsight Workspaces and Cortex Code	Native in Snowsight Workspaces and Cortex Code
DDL variants	Only non-destructive changes via DEFINE (=CREATE OR ALTER) leaving all attachments intact	Any CREATE OR REPLACE execution loses all attached policies, tags, grants, expectations -> need to be reapplied
Scope beyond Snowflake	Snowflake only	Any supported SQL database or data platform (e.g. PostgreSQL)

© 2026 Snowflake Inc. All Rights Reserved